

# bulk\_register\_aips ❖❖❖ Piano di Test

Direzione ICT, Università❖❖ degli Studi di Milano

Luglio 2026

## Contents

|                                                                                                          |          |
|----------------------------------------------------------------------------------------------------------|----------|
| <b>Piano di test — bulk_register_aips.py</b>                                                             | <b>1</b> |
| Test 1 — AIP già registrati dalla stessa istanza, spostati in un'altra directory . . . . .               | 1        |
| Test 2 — AIP ereditati da un'altra installazione Archivematica (DB perso, resta solo lo store) . . . . . | 2        |
| Test 3 — AIP da a3m/DART3 in cartella piatta (non quadrant-sharded) . . . . .                            | 4        |
| Checklist finale prima di applicare i test a dati reali di produzione . . . . .                          | 5        |

## Piano di test — bulk\_register\_aips.py

Tre scenari operativi distinti, pensati per verificare comportamenti diversi dello script. Vanno eseguiti **sempre su copie dei dati**, mai sugli AIP originali/di produzione, e **sempre partendo da --dry-run** prima di un'esecuzione reale.

Prerequisiti comuni a tutti i test:

- una cartella di lavoro separata dai dati di produzione (es. /tmp/test\_bulk\_register/);
- un TARGET\_LOCATION\_UUID di **test** nella Storage Service (non quella di produzione), oppure massima cautela se si è costretti a usare quella reale;
- accesso in lettura alla Storage Service per verificare via API lo stato prima/dopo (GET /api/v2/file/<uuid>/, GET /api/v2/file/?package\_type=AIP);
- una copia di .env puntata all'ambiente di test.

---

## Test 1 — AIP già registrati dalla stessa istanza, spostati in un'altra directory

### Obiettivo

Verificare che lo script **non tenti mai di registrare di nuovo** un AIP che risulta già noto alla Storage Service, anche se lo si ritrova fisicamente in una directory diversa da quella originale. Questo è un test di sicurezza/guardrail: lo script è pensato per registrare AIP **orfani** (non ancora tracciati), non per gestire lo spostamento di un AIP già tracciato — quell'operazione è di competenza della Storage Service stessa (o di un tool dedicato, da valutare separatamente se serve davvero).

### Istruzioni operative

1. Scegli un AIP che sai per certo essere già registrato normalmente (es. prodotto e ingestito dalla tua istanza Archivematica in uso). Annotane l'UUID, ad esempio consultan-

dolo dal Dashboard o con:

```
curl -k -H "Authorization: ApiKey archivematica:LA_TUA_API_KEY" \
  "https://192.168.139.39:8000/api/v2/file/?package_type=AIP&limit=1"
```

2. **Copia** (non spostare) il file fisico di quell'AIP dentro una nuova cartella di test, es.:

```
mkdir -p /tmp/test_bulk_register/test1_gia_registrati
cp /percorso/reale/<nome>-<uuid>.7z /tmp/test_bulk_register/test1_gia_registrati/
```

3. Esegui lo script in dry-run puntando alla nuova cartella:

```
python3 bulk_register_aips.py \
  --scan-dir /tmp/test_bulk_register/test1_gia_registrati \
  --target-location-uuid <UUID_LOCATION_TEST> \
  --dry-run
```

### Risultato atteso

- Nel conteggio "AIP trovati fisicamente" l'AIP compare (1).
- Nel conteggio "AIP già registrati nella Storage Service" viene incluso nel totale già noto.
- **"AIP orfani da registrare" deve essere 0** per questo AIP: nessuna riga di log [DRY-RUN] Registrerei ... per il suo UUID.
- Nel file di report JSON, la sua chiave UUID **non** compare in results (perché non è stato considerato orfano).

### Se il test fallisce

Se invece lo script propone di registrarlo, significa che `get_registered_aip_uuids()` non lo ha trovato tra i registrati (es. problema di paginazione dell'API, o l'AIP non è davvero registrato come pensavi) — va investigato prima di procedere con qualunque altro test.

### Nota

Se l'obiettivo reale è **spostare fisicamente** un AIP già tracciato verso una nuova directory (non solo verificare il comportamento di sicurezza), questo script non è lo strumento giusto: serve un'azione dedicata lato Storage Service (spostamento di location/rilocazione del package), che è un'operazione diversa dalla registrazione di un AIP orfano. Se ti serve, possiamo valutare uno script a parte per quel caso specifico.

---

## Test 2 — AIP ereditati da un'altra installazione Archivematica (DB perso, resta solo lo store)

### Obiettivo

Verificare il caso d'uso principale dello script: una cartella con struttura a quadranti (pairtree) già esistente, prodotta da un'altra istanza Archivematica di cui non esiste più il database, va registrata in una Storage Service diversa **senza** che i file vengano spostati inutilmente (dato che la struttura è già corretta).

### Istruzioni operative

1. Prepara una copia della cartella (o di un sottoinsieme rappresentativo, es. 3-5 AIP) dello store ereditato, mantenendo intatta la struttura a quadranti:

```
mkdir -p /tmp/test_bulk_register/test2_ereditati
cp -r /percorso/store/ereditato/1a2b /tmp/test_bulk_register/test2_ereditati/
# ripeti per ciascuna cartella di primo livello (quadrante) che contiene
# gli AIP scelti per il test
```

**Importante:** --scan-dir dovrà puntare esattamente alla **radice** di questa struttura (la cartella che contiene direttamente le sottocartelle a 4 caratteri esadecimali), non a una cartella che la contiene con un livello di annidamento in più — altrimenti l’auto-detect “già a quadranti” non riconoscerà la struttura (vedi sezione 3 del manuale).

2. Prima di lanciare lo script su tutti gli AIP, verifica lo schema a quadranti della TUA Storage Service di destinazione confrontandolo con un AIP già registrato normalmente lì:

```
curl -k -H "Authorization: ApiKey archivematica:LA_TUA_API_KEY" \
  "https://192.168.139.39:8000/api/v2/file/<uuid-noto-gia-registrato>/"
```

e controlla il campo `current_path` restituito.

3. Esegui il dry-run:

```
python3 bulk_register_aips.py \
  --scan-dir /tmp/test_bulk_register/test2_ereditati \
  --target-location-uuid <UUID_LOCATION_DESTINAZIONE> \
  --dry-run
```

## Risultato atteso

- Tutti gli AIP compaiono come “orfani da registrare”.
- Per ciascuno, il log mostra “**senza spostamento (già a quadranti)**” (`will_move=False`).
- `uuid_source` è filename per gli AIP con nome standard `<nome>-<uuid>.<ext>`.
- Nessun conflitto UUID (`conflicts` vuoto), a meno che i file non siano stati rinominati durante il recupero.

## Prosecuzione (dopo il dry-run positivo)

4. Esegui la registrazione reale (senza --dry-run) sulla cartella di test:

```
python3 bulk_register_aips.py \
  --scan-dir /tmp/test_bulk_register/test2_ereditati \
  --target-location-uuid <UUID_LOCATION_DESTINAZIONE>
```

5. Verifica via API che gli AIP risultino ora registrati con il `current_path` atteso:

```
curl -k -H "Authorization: ApiKey archivematica:LA_TUA_API_KEY" \
  "https://192.168.139.39:8000/api/v2/file/<uuid-registrato-ora>/"
```

6. Rigenera l’indice del Dashboard e verifica che compaiano nella ricerca “Archival Storage”:

```
manage.py rebuild_aip_index_from_storage_service -u <uuid-registrato-ora>
```

Solo dopo aver validato questi passaggi su un piccolo sottoinsieme, procedi con l’intera cartella ereditata.

## Test 3 — AIP da a3m/DART3 in cartella piatta (non quadrant-sharded)

### Obiettivo

Verificare il fallback di identificazione via METS (per nomi non standard), il cross-check di coerenza UUID (per nomi che invece un UUID ce l'hanno), e il corretto calcolo + spostamento fisico verso lo schema a quadranti di destinazione.

### Istruzioni operative

1. Prepara una cartella piatta di test con alcuni AIP prodotti da a3m/DART3:

```
mkdir -p /tmp/test_bulk_register/test3_a3m_dart3
cp /percorso/a3m-o-dart3/output/*.7z /tmp/test_bulk_register/test3_a3m_dart3/
```

2. Controlla manualmente i nomi dei file copiati e dividili mentalmente in due gruppi, per interpretare correttamente i risultati:

- quelli il cui nome **contiene già** un UUID nel formato <nome>-<uuid>.<ext>;
- quelli il cui nome **non** lo contiene (naming diverso, es. solo l'UUID senza prefisso, o un nome descrittivo senza UUID).

3. Esegui il dry-run:

```
python3 bulk_register_aips.py \
  --scan-dir /tmp/test_bulk_register/test3_a3m_dart3 \
  --target-location-uuid <UUID_LOCATION_TEST> \
  --dry-run
```

### Risultato atteso

- Per gli AIP del primo gruppo (nome con UUID): `uuid_source: filename`, e se il METS interno conferma lo stesso UUID, `uuid_mismatch: False` (nessun conflitto).
- Per gli AIP del secondo gruppo (nome senza UUID): `uuid_source: mets` — l'UUID è stato letto aprendo l'archivio, non dal nome del file. Se per qualcuno di questi lo script non trova nulla (perché il METS non è leggibile o manca), quell'AIP **non comparirà affatto** tra i risultati: controlla il conteggio "AIP trovati fisicamente" e confrontalo col numero di file effettivamente presenti nella cartella per accorgertene.
- Per **tutti** gli AIP di questo test: log **"CON spostamento fisico -> quadrant path"** (`will_move=True`), dato che partono da una struttura piatta. Controlla che il `current_path` calcolato nel log sia coerente con lo schema a quadranti (blocco per blocco di 4 caratteri dell'UUID + cartella con UUID completo + nome file).
- Se `.7z` è tra i formati testati e il comando `7z` non è installato sul server, vedrai un `[WARN]` comando `'7z'` non trovato...: in quel caso il test coprirà solo l'identificazione da filename, non il fallback METS né il cross-check per quei file specifici — installa `p7zip-full` per un test completo.

### Prosecuzione (dopo il dry-run positivo)

4. Esegui la registrazione reale su questa cartella di test **verificando prima che <UUID\_LOCATION\_TEST> non sia la location di produzione:**

```
python3 bulk_register_aips.py \
  --scan-dir /tmp/test_bulk_register/test3_a3m_dart3 \
  --target-location-uuid <UUID_LOCATION_TEST>
```

5. Verifica che i file siano stati effettivamente spostati (fisicamente, sul filesystem della location di test) nella struttura a quadranti attesa, e che il record nella Storage Service abbia lo stesso `current_path`.
  6. Se anche gli AIP con eventuale conflicts (mismatch UUID) sono presenti nella cartella di test, verificane manualmente uno: apri l'archivio, controlla a mano il METS, e conferma se l'UUID "giusto" è davvero quello del METS (come assume lo script) o se il caso specifico richiede un'altra interpretazione.
- 

## Checklist finale prima di applicare i test a dati reali di produzione

- ☐ Ogni test è stato eseguito prima in `--dry-run`.
- ☐ Ogni test è stato eseguito su **copie**, mai sugli originali.
- ☐ Il `TARGET_LOCATION_UUID` usato nei test non è quello di produzione (o, se lo è per forza, il rischio è stato valutato consapevolmente).
- ☐ Per il Test 2, il `current_path` calcolato è stato confrontato con quello di un AIP già registrato normalmente sulla Storage Service di destinazione.
- ☐ Per il Test 3, eventuali conflitti UUID sono stati verificati manualmente prima di decidere come trattarli.
- ☐ Dopo ogni registrazione reale, è stato lanciato `manage.py rebuild_aip_index_from_storage_service` per verificare la comparsa nel Dashboard.